Bull Dog Solution — Órdenes de servicio

Sistema web responsivo para preparar, asignar, ejecutar y auditar órdenes de instalación, revisión y desinstalación de equipos en unidades vehiculares.

> Este README describe el estado objetivo del producto una vez concluido el MVP. La implementación se encuentra organizada en ROADMAP.md y su definición técnica completa está en SPECS.md.

La documentación del estado técnico implementado, incluidos los diagramas de arquitectura y entidad-relación, está disponible en docs/technical/README.md.

Descripción

La plataforma sustituye el flujo operativo realizado en AppSheet y reúne en un solo lugar el trabajo administrativo y de campo:

- Los administradores crean órdenes, seleccionan clientes, registran unidades, asignan equipos y eligen al técnico responsable.
- Los técnicos consultan sus servicios, verifican equipos, documentan cada unidad con checklist y fotografías, reportan discrepancias y obtienen la firma del cliente.
- Google Sheets conserva la autoridad sobre los catálogos de clientes y equipos.
- PostgreSQL mantiene los datos transaccionales, el historial y el estado de sincronización.
- Las fotografías y firmas se almacenan de forma privada en un servicio compatible con Amazon S3.

Funcionalidades principales

Administración

- Autenticación segura y administración local de usuarios y técnicos.
- Creación de órdenes con varias unidades y varios equipos por unidad.
- Consulta de clientes y equipos directamente desde Google Sheets.
- Asignación y reasignación de técnicos.
- Seguimiento del estado y progreso de cada servicio.
- Detección visible de conflictos cuando una fila de Sheets cambia externamente.
- Consulta de evidencias, firma, discrepancias, auditoría y sincronización.
- Reporte PDF de cada orden completada con portada, detalle por unidad/equipo y fotografías en formato amplio.
- Reintento controlado de operaciones fallidas con Google Sheets.

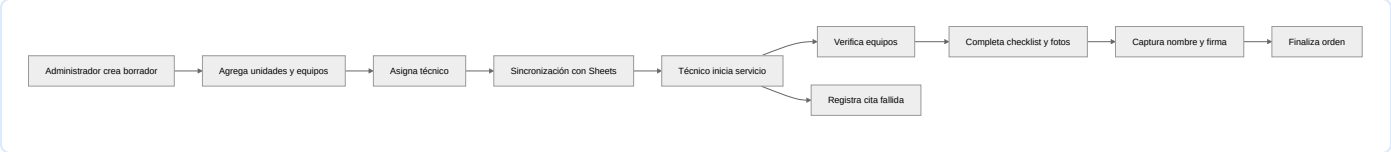
Operación técnica

- Vista mobile-first de las órdenes asignadas.
- Verificación individual de equipos.
- Reporte de diferencias de ID, IMEI, serie o unidad sin bloquear el trabajo.
- Checklist posterior al servicio para tablero, cristales, accesorios, molduras y carrocería.
- Estados `Bien`, `Mal` y `No aplica`, con observación obligatoria para elementos en mal estado.
- Captura de al menos una fotografía por categoría.
- Firma única del cliente y registro del nombre de la persona firmante.
- Cierre normal con validación integral o cierre simplificado como cita fallida.

Seguridad y trazabilidad

- Access y refresh tokens JWT con rotación y revocación de sesiones.
- Autorización por rol y pertenencia de la orden.
- Validación de todas las entradas mediante Valibot.
- Almacenamiento privado y URLs prefirmadas de corta duración.
- Auditoría de operaciones administrativas y cambios relevantes.
- Idempotencia, concurrencia optimista y outbox para efectos externos.
- Logs estructurados con redacción de secretos y datos sensibles.

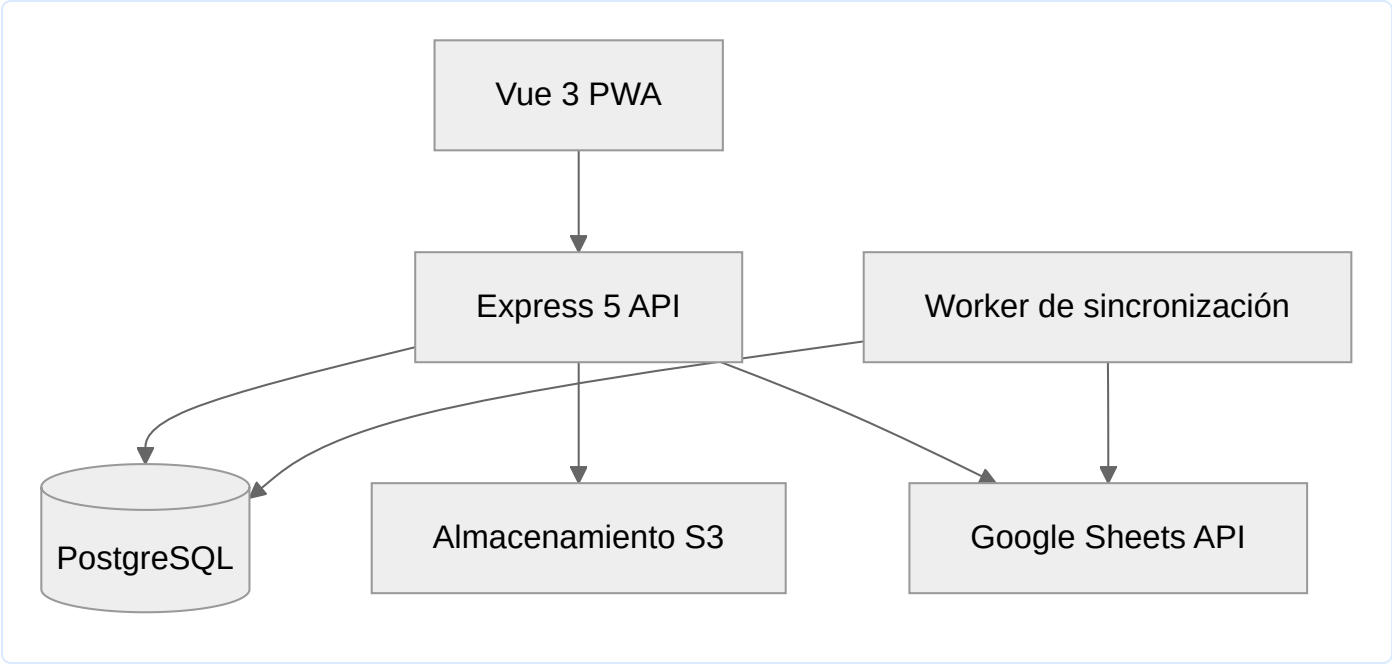
Flujo del servicio



Una discrepancia puede registrarse durante la verificación y no impide finalizar la orden cuando los demás requisitos están completos.

Arquitectura

El proyecto utiliza un monolito modular en un monorepo. El frontend y la API se desarrollan y construyen por separado, mientras que las reglas de negocio permanecen en el backend.



Capa	Tecnologías
Lenguaje	TypeScript
Frontend	Vue 3, Vite, Vue Router, Pinia, TanStack Query
Interfaz	Naive UI, Tailwind CSS, VueUse
API	Node.js, Express 5, Valibot
Persistencia	PostgreSQL, Prisma
Autenticación	JWT, Argon2id
Integraciones	Google Sheets API, API compatible con Amazon S3
Documentación	OpenAPI 3.1, Swagger UI
Pruebas	Vitest, Supertest, Playwright
Infraestructura	Docker, Docker Compose, pnpm

Estructura del repositorio

```
.
├── apps/
│   ├── api/                # API Express y módulos de negocio
│   └── web/                # Aplicación Vue y PWA
├── packages/
│   ├── contracts/          # DTOs y esquemas Valibot compartidos
│   ├── eslint-config/      # Configuración común de lint
│   └── typescript-config/  # Configuración común de TypeScript
├── prisma/
│   ├── migrations/         # Historial de migraciones
│   ├── schema.prisma       # Modelo de datos
│   └── seed.ts             # Datos iniciales de desarrollo
├── infra/
│   └── docker/             # Archivos de infraestructura local
├── docker-compose.yml
├── pnpm-workspace.yaml
├── ROADMAP.md
└── SPECS.md
```

Los módulos siguen slicing vertical. Cada dominio contiene sus rutas, controladores, servicios, persistencia, contratos y pruebas. Las integraciones con Google Sheets y S3 se consumen a través de interfaces reemplazables.

Estados de una orden

Una orden normal sigue este recorrido:

```
DRAFT → ASSIGNED → IN_PROGRESS → COMPLETED
                 → REQUIRES_CORRECTION → IN_PROGRESS
       → CANCELLED
```

Una cita fallida utiliza el flujo:

```
ASSIGNED → IN_PROGRESS → FAILED_APPOINTMENT
```

Para completar un servicio normal, todas las unidades deben tener sus equipos verificados, checklist completo y fotografías requeridas. La orden también debe incluir el nombre y la firma del cliente. Una cita fallida requiere únicamente un motivo.

Integración con Google Sheets

Google Sheets permanece como fuente oficial de los catálogos externos:

- `Clientes`: lectura de `IDCliente` y `Cliente`.
- `SALIDAS`: lectura del catálogo de equipos y actualización limitada de su asignación.

La aplicación solo puede escribir las columnas autorizadas:

Columna	Contenido
`J`	Cliente
`L`	Unidad
`M`	Marca/modelo de la unidad
`N`	Año
`O`	Número de serie
`P`	Tipo de servicio, cuando el negocio lo habilite

Las columnas `A`, `I` y `K` nunca son modificadas. Antes de cada escritura, la API vuelve a localizar el equipo por ID e IMEI, compara el snapshot conocido y detiene la operación con un conflicto HTTP `409` si detecta cambios externos.

Requisitos

- Node.js 24.20.0 LTS, fijado en `.nvmrc` y `.node-version`.
- pnpm 11.9.0 mediante Corepack.
- Docker y Docker Compose.
- Documento de Google Sheets de desarrollo compartido con una cuenta de servicio.
- Credenciales de Google Sheets y almacenamiento S3 para entornos no locales.

Puesta en marcha local

1. Activar la versión fijada de pnpm e instalar las dependencias:

```
corepack enable
pnpm install --frozen-lockfile
```

2. Crear la configuración local a partir del archivo de ejemplo:

```
cp .env.example .env
```

3. Para trabajar con recarga en caliente, levantar PostgreSQL y MinIO:

```
docker compose up -d postgres minio minio-init
```

4. Aplicar las migraciones y cargar datos iniciales:

```
pnpm prisma:migrate
pnpm prisma:seed
```

El seed local crea `admin@example.invalid` y `technician@example.invalid`. Cambia `SEED_ADMIN_PASSWORD` antes de ejecutarlo fuera de un entorno local; en producción es obligatorio.

5. Iniciar API y web:

```
pnpm dev
```

La aplicación web queda disponible por defecto en `http://localhost:5173` y la API en `http://localhost:3000`. `VITE_DEV_PORT`, `VITE_API_PROXY_TARGET` y `PORT` permiten usar puertos diferentes sin modificar código.

Como alternativa, el stack completo compilado se levanta con:

```
docker compose up --build -d
```

En esta modalidad, la web se publica por defecto en `http://localhost:4173`, la API en `http://localhost:3000` y la consola de MinIO en `http://localhost:9001`. Los puertos pueden sobrescribirse en `.env` con las variables documentadas en `.env.example`.

Configuración

La aplicación valida su configuración al iniciar y se detiene si falta una variable obligatoria. Los grupos principales son:

Grupo	Variables principales
Aplicación	`NODE_ENV`, `PORT`, `APP_ORIGIN`, `API_ORIGIN`, `TRUST_PROXY`
Base de datos	`DATABASE_URL`, `DIRECT_URL`
JWT	Claves pública/privada, issuer, audience y TTL de cada token
Cookies	Dominio, modo seguro y política SameSite
Objetos	Endpoint, región, bucket y credenciales S3
Google Sheets	ID del documento, cuenta de servicio y nombres de pestañas
Límites	Tamaños de evidencia/firma, TTL de URLs y caché de catálogos
Resiliencia	Timeouts, reintentos, circuit breaker y frecuencia del outbox
Logs	`LOG_LEVEL`

Los secretos no deben guardarse en el repositorio. En producción se utilizan identidades administradas o un gestor de secretos cuando el proveedor lo permita.

Comandos disponibles

Comando	Descripción
`pnpm dev`	Inicia API y web en desarrollo
`pnpm build`	Construye todas las aplicaciones y paquetes
`pnpm format`	Formatea los archivos del repositorio
`pnpm format:check`	Verifica el formato sin escribir cambios
`pnpm lint`	Ejecuta el análisis estático
`pnpm typecheck`	Comprueba TypeScript estricto
`pnpm test`	Ejecuta pruebas unitarias y de integración
`pnpm test:e2e`	Ejecuta escenarios Playwright
`pnpm docs:build`	Genera OpenAPI 3.1
`pnpm docs:validate`	Valida el contrato de la API
`pnpm prisma:migrate`	Aplica migraciones de base de datos
`pnpm prisma:seed`	Carga datos iniciales de desarrollo
`pnpm prisma:validate`	Valida la configuración y el esquema Prisma

API y documentación

La API utiliza el prefijo `/api/v1` y responde en JSON con un `requestId` para trazabilidad.

- Especificación OpenAPI: `/api/v1/openapi.json`.
- Swagger UI: `/api/docs`.
- Estado de proceso: `/health/live`.
- Estado de dependencias: `/health/ready`.

Los endpoints abarcan autenticación, usuarios, técnicos, catálogos externos, órdenes, unidades, equipos, checklist, evidencias, firma, auditoría y sincronización.

Pruebas

La estrategia combina:

- Pruebas unitarias para validaciones, transiciones, autorización, rotación de tokens e identificación de equipos.
- Pruebas de integración para Express, PostgreSQL, almacenamiento de objetos, Sheets simulado, outbox e idempotencia.

- Pruebas de contrato para comparar OpenAPI con respuestas reales de la API.
- Pruebas end-to-end de los recorridos completos de administrador y técnico.

Los escenarios automatizados nunca escriben en la hoja de producción.

Despliegue

El producto se despliega con entornos separados de desarrollo, staging y producción:

- Frontend estático servido mediante CDN o reverse proxy.
- API ejecutada como contenedor Node.js.
- PostgreSQL administrado con respaldos automáticos y restauración probada.
- Bucket S3 privado con bloqueo de acceso público.
- Reverse proxy con TLS y, preferentemente, API bajo el mismo dominio público.
- Google Sheets distinto para staging y producción.

El pipeline ejecuta instalación reproducible, lint, typecheck, pruebas, generación de OpenAPI, builds, escaneos, migraciones controladas, despliegue y smoke tests.

Alcance del MVP

El MVP no incluye operación offline completa, aplicaciones móviles nativas, edición de los catálogos externos, facturación, cuentas bancarias, tarifas de técnicos ni microservicios. Tampoco modifica automáticamente `Fecha de salida`, `Estado` o `Estado del servicio` en Google Sheets.

Documentación del proyecto

- Especificación técnica
- Ruta de implementación